CAMPUSCARE – SMART CAMPUS SERVICE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the course

SMART CAMPUS SERVICE MANAGEMENT SYSTEM

BACHELOR OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

K L (Deemed to be) UNIVERSITY

Academic Year 2026–27

Team 22 — Section 7

Param Manasvi — 2520030072

Ch. Shravya Chowdary — 2520030322

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty and department for providing the academic environment and project-based learning opportunity to design and document CampusCare, a Smart Campus Service Management System. The project connects software engineering concepts with a practical campus service workflow.

We acknowledge the support of classmates, friends and family during planning, implementation, debugging and documentation. The project provided practical exposure to React-based interfaces, Java and Spring Boot REST services, relational database design, authentication, role-based access and service request management.

Team 22 — Section 7
Param Manasvi — 2520030072
Ch. Shravya Chowdary — 2520030322

ABSTRACT

CampusCare is a web-based Smart Campus Service Management System designed to make campus issue reporting and resolution more organized and transparent. Students can submit requests for problems involving facilities such as electricity, plumbing, cleanliness, maintenance and internet connectivity. Requests include location details, priority, category and status so the relevant service team can act on them.

The application uses React 18 and Vite for the frontend, Java 17 with Spring Boot 3.3.5 for the backend, and MySQL for persistent storage. Spring Data JPA/Hibernate handles database access, while Spring Security and JSON Web Tokens support authenticated, role-aware access. The system includes student request tracking, technician status updates, administrator assignment and management functions, request history, feedback, overdue alerts, keyword-based category suggestion and an FAQ chatbot.

The database separates users, departments, categories, service requests, request updates, feedback and chatbot FAQs. Foreign keys, unique constraints, a rating check and request indexes support data integrity and retrieval. The project demonstrates how a structured digital workflow can improve accountability and visibility in campus maintenance operations. The testing chapter provides functional and security checks to run against the application; actual test outcomes should be added after execution.

INDEX

S. No.	Chapter / Section	Contents
1	Introduction	Background, problem statement, objectives, scope and users
2	System Requirements	Hardware, software and development requirements
3	Technology Stack	Frontend, backend, database and security
4	System Architecture	Architecture layers and request lifecycle
5	Database Design	Entities, relationships, constraints and seed data
6	Implementation	Authentication, requests, admin, classifier and chatbot
7	Features	Student, technician, admin and smart assistance
8	Testing and Validation	Functional, database and security test cases
9	Deployment and Execution	Database, backend and frontend setup
10	Challenges and Limitations	Known limitations and mitigation
11	Future Enhancements	Classification, notifications and analytics
12	Conclusion	Summary and learning outcomes
13	References	Project files and technology documentation
14	Appendices	Project structure, endpoints and user guide

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

A campus contains many shared facilities such as classrooms, laboratories, washrooms, electrical systems, Wi-Fi networks and common areas. When a facility fails, students and staff need a clear way to report the issue and follow its progress. Informal reporting through calls or messages can cause delays, duplicated complaints and a lack of visibility about responsibility.

1.2 Problem Statement

The project addresses the absence of a single, structured platform for reporting, assigning, tracking and resolving campus service issues. Requests need to be categorized, routed to the correct department, assigned to a responsible technician and recorded throughout their lifecycle. Administrators also need an overview of request volume, pending work and overdue issues.

1.3 Proposed Solution

CampusCare is a web-based Smart Campus Service Management System. It provides role-based screens for students, technicians and administrators. Users can submit service requests with campus location details, track status changes and provide feedback after resolution. Administrators can manage users, categories and departments, assign work and review dashboard statistics. A keyword-based classifier suggests a category, while a FAQ chatbot answers common campus questions.

1.4 Objectives

- Provide a central location for campus service requests.
- Support authentication and role-based access.
- Record request status changes in a timeline.
- Help administrators assign requests to departments and technicians.
- Suggest categories from request descriptions.
- Highlight requests that remain unresolved for more than 72 hours.
- Collect ratings and comments after resolution.
- Provide a maintainable React, Spring Boot and MySQL application.

1.5 Scope

The implementation covers authentication, service-request creation and tracking, request history, technician assignment, department/category management, user management, feedback, dashboard analytics, overdue alerts and FAQ-based assistance. The provided code uses a local keyword-scoring classifier rather than a trained machine-learning model. External integrations such as campus ERP, SMS/email gateways, payment services and production hosting are outside the documented implementation.

1.6 Target Users

- Student: submits a request, checks progress and submits feedback.
- Technician: views assigned work, updates status and adds resolution remarks.
- Staff / Department Head: role options are represented in the access model; available actions depend on configured permissions.
- Administrator: reviews statistics, assigns requests and manages users, categories and departments.

1.7 Significance

A shared workflow improves visibility and accountability. Structured records make it easier to identify recurring issues, understand resolution delays and retain a history of actions. Location fields such as building, floor and room help service teams find the reported problem.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.1 Hardware Requirements

- Computer or laptop capable of running a modern browser and local development services.
- At least 4 GB RAM recommended; 8 GB or more is preferable when running MySQL, Java and Node.js together.
- Network connection for package installation and browser access.

2.2 Software Requirements

- Operating system: Windows, Linux or macOS.
- Java Development Kit (JDK) 17 or later.
- Maven 3.8 or later.
- MySQL 8.x database server.
- Node.js 18 or later and npm.
- Modern web browser.
- Code editor such as Visual Studio Code or IntelliJ IDEA.

2.3 Development Requirements

The frontend is run with Vite on port 5173. The Spring Boot backend listens on port 8080. The datasource configuration in the provided application.properties uses port 3307 by default, so the MySQL service and DB_URL must match that setting (or be changed to the local MySQL port). The database schema and seed scripts are included in the project ZIP.

2.4 Constraints

- Database credentials and JWT secrets must be configured securely before deployment.
- The backend depends on a running MySQL instance.
- The keyword classifier and FAQ matching are rule-based and may not understand every phrasing.
- The provided archive includes dependency/build artifacts; a clean deployment should avoid committing node_modules and generated target files.

CHAPTER 3 – TECHNOLOGY STACK

3.1 Frontend

React 18 provides component-based user interfaces. Vite supplies the development server and build process. The frontend code is primarily in App.jsx, api.js and styles.css. Recharts is included for charting and dashboard visualizations.

3.2 Backend

Java 17 and Spring Boot 3.3.5 implement the REST API. Spring Web provides HTTP endpoints; Spring Data JPA and Hibernate map Java entities to relational tables; Bean Validation supports input validation; Spring Security applies authentication and authorization rules.

3.3 Database

MySQL stores users, departments, categories, service requests, request updates, feedback and chatbot FAQs. The schema defines primary keys, foreign keys, uniqueness constraints, a feedback rating check and indexes for common request queries.

3.4 Security

Passwords are stored as BCrypt hashes in the seed data. The backend uses JSON Web Tokens (JWT) for authenticated requests, with a JWT filter and role-aware security configuration. The frontend API client stores/attaches the token for protected API calls. For production, secrets should be supplied through environment variables and changed from development defaults.

3.5 Build and Configuration

Maven builds and runs the backend. npm installs frontend packages and runs the Vite server. The project includes .env.example and application.properties configuration. The README documents database initialization, backend startup, frontend startup and key endpoints.

3.6 Main Dependencies

- Spring Boot Starter Web
- Spring Boot Starter Data JPA
- Spring Boot Starter Validation
- Spring Boot Starter Security
- MySQL Connector/J
- JJWT 0.12.6 (API, implementation and Jackson integration)
- React 18, React DOM, Vite and Recharts

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 Architecture Overview

CampusCare follows a three-layer web architecture: a React frontend, a Spring Boot REST backend and a MySQL database. The frontend communicates with the backend through /api endpoints. Authentication responses provide a JWT, which is attached to later protected requests.

4.2 Logical Architecture

```
Browser / React UI
  ■■■ Authentication and role-based dashboards
  ■■■ Student request submission and tracking
  ■■■ Technician work queue and status updates
  ■■■ Admin analytics and management pages
  ■■■ FAQ chatbot widget
            ■ HTTP / JSON + JWT
            ▼
Spring Boot REST API (port 8080)
  ■■■ Web controllers: Auth, Request, Admin, User, Chatbot
  ■■■ Services: request workflow and category classifier
  ■■■ Security: Spring Security, JWT filter and JWT utility
  ■■■ Repositories: Spring Data JPA
  ■■■ Domain models and validation
            ■ JDBC / JPA
            ▼
MySQL database: users, requests, departments, categories,
request_updates, feedback and chatbot_faqs
```

4.3 Backend Layers

Controllers receive HTTP requests and return responses. Service classes contain workflow logic such as creating or updating requests and suggesting categories. Repository interfaces perform database operations through JPA. Model classes represent persisted entities and role/status values. Security configuration and JWT utilities protect authenticated endpoints.

4.4 Request Lifecycle

The documented workflow is OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED. Request updates are recorded in the request_updates table so users and administrators can inspect the history. A request may be categorized and assigned to a department and technician as part of the workflow.

4.5 Deployment View

For local development, the browser opens the Vite frontend at http://localhost:5173, while the Spring Boot API runs at http://localhost:8080. Vite is configured to proxy API requests to the backend. MySQL runs separately and must be reachable using the configured JDBC URL.

CHAPTER 5 – DATABASE DESIGN

5.1 Main Entities

- Users: identity, email, password hash, role and optional department.
- Departments: campus service teams such as Maintenance, Electrical, Plumbing, Housekeeping and IT & Network.
- Categories: issue categories with an optional default department.
- Service Requests: title, description, location, priority, status, requester, assigned technician, category and department.
- Request Updates: status, comment, updater and timestamp for the request timeline.
- Feedback: request-linked rating and comments.
- Chatbot FAQs: question, answer and matching keywords.

5.2 Relationships

A user can create many service requests. A request can have multiple history updates and at most one feedback record in the supplied schema. Requests may reference a category, department and assigned technician. A category may map to a default department. Department membership is optional for users.

5.3 Integrity and Indexing

Primary keys identify records. Foreign keys connect related entities and protect referential integrity. User email and department/category names are unique. The feedback rating is constrained to the range 1–5. Indexes on request status, requester and assigned technician support common filtering and dashboard operations.

5.4 Seed Data

The seed script creates common campus departments and categories, plus demo administrator, technician and student records. It also inserts sample requests for an electrical issue, a plumbing issue and a Wi-Fi issue. Demo credentials are intended only for local demonstration and must be changed before real use.

5.5 Data Model Summary

The database design separates user identity, service request details, status history and feedback. This avoids placing all information in one large table and supports querying requests by owner, status, technician, department and category.

CHAPTER 6 – IMPLEMENTATION

6.1 Authentication

The AuthController exposes registration, login and current-user endpoints. Login returns a token for authenticated sessions. The security layer validates JWTs and applies access rules. Password hashing is handled with BCrypt rather than storing plain-text passwords.

6.2 Service Request Management

The RequestController exposes endpoints for creating requests, listing requests according to the caller's role, retrieving a single request, reading request history, changing status, assigning a department or technician, suggesting a category and submitting/retrieving feedback. RequestService contains the primary workflow logic.

6.3 Automatic Category Suggestion

ClassifierService evaluates words and phrases in the request description and proposes a likely category. The response is a suggestion to support data entry; it should not be described as a trained AI model. The request record tracks whether a category was automatically assigned/suggested.

6.4 Admin Dashboard and Management

AdminController supplies dashboard statistics and overdue requests, including requests unresolved for more than 72 hours. Category and department CRUD endpoints allow administrators to maintain the classification and routing lists. UserController provides user/technician listing and staff-management operations.

6.5 Technician Workflow

Technicians can view requests assigned to them, update status and add resolution remarks. The history records who made a change and when, helping preserve accountability.

6.6 FAQ Chatbot

ChatbotController exposes an endpoint to query the FAQ bot and an endpoint to list available FAQs. The matching approach scores keyword overlap against stored FAQ keywords and returns the best matching answer. The README describes seven common campus questions as the initial knowledge set.

6.7 Frontend Integration

The React application contains authentication screens, a shared application shell, request tables, request details/timeline, request submission, admin dashboard, user management, category management, department management and a chatbot widget. api.js centralizes API calls and attaches JWT information. styles.css controls layout and responsive presentation.

6.8 API Summary

- POST /api/auth/register — register a student.
- POST /api/auth/login — authenticate and receive a token.
- GET /api/auth/me — retrieve the current user profile.
- POST /api/requests — create a service request.
- GET /api/requests — list requests according to role and filters.
- GET /api/requests/{id}/history — retrieve request timeline.
- PUT /api/requests/{id}/status — update request status.
- PUT /api/requests/{id}/assign-department — assign department (admin).
- PUT /api/requests/{id}/assign-technician — assign technician (admin).
- POST /api/requests/suggest-category — request category suggestion.

• GET /api/admin/stats and GET /api/admin/overdue — administrative summaries.

• GET /api/chatbot/ask?q=... — query FAQ assistance.

CHAPTER 7 – FEATURES

7.1 Student Features

- Create an account and log in.
- Submit a campus service request with title, description and location details.
- View request status and status history.
- Use category suggestions while describing an issue.
- Submit a star rating and comments after resolution.
- Ask common campus questions through the FAQ chatbot.

7.2 Technician Features

- View assigned service requests.
- Update the work status as the issue progresses.
- Add remarks to describe work or resolution.
- Maintain a recorded history of status changes.

7.3 Administrator Features

- Review request statistics in the dashboard.
- Identify overdue requests.
- Assign requests to departments and technicians.
- Create, update and delete categories and departments.
- Manage user and technician records.
- Search and filter request tables.

7.4 Smart Assistance

- Keyword-based category suggestion helps route requests.
- FAQ chatbot answers common campus service questions.
- Overdue detection flags requests unresolved for more than 72 hours.
- Location fields include building, floor and room to improve issue localization.

7.5 Request Statuses

OPEN: request has been submitted. ASSIGNED: responsibility has been allocated. IN_PROGRESS: work is underway. RESOLVED: the service issue has been addressed. CLOSED: the workflow is completed.

CHAPTER 8 – TESTING AND VALIDATION

8.1 Testing Approach

The source archive was reviewed to document the implemented modules, endpoint structure, database constraints and startup configuration. The following are recommended functional and integration tests. They are test cases to execute in the running environment, not a claim that every test has already passed.

8.2 Functional Test Cases

- Authentication: valid login returns a token; invalid credentials are rejected.
- Registration: valid student data creates an account; duplicate email is rejected.
- Request creation: a valid request is saved with the requester and timestamp.
- Role-based listing: students see permitted requests; technicians see assigned work; administrators can review overall requests.
- Status update: status changes are persisted and a history record is created.
- Assignment: an administrator can assign a department and technician.
- Category suggestion: descriptions containing known issue keywords return a plausible category.
- Feedback: a resolved request accepts a rating from 1 to 5 and optional comments.
- Overdue detection: unresolved requests older than 72 hours are returned by the admin endpoint.
- FAQ chatbot: a query matching stored keywords returns an answer.

8.3 Database Validation

Verify foreign-key relationships, unique email/category/department values, feedback rating range, and request indexes. Confirm that deleting or changing related records follows the configured constraints. Confirm that schema.sql is run before seed.sql when initializing a new database.

8.4 Security Validation

Verify that protected endpoints reject missing or invalid JWTs and that role-restricted operations cannot be performed by unauthorized users. Replace development credentials and the JWT secret before any deployment beyond a local demonstration.

8.5 Test Result Recording

For final submission, record each test’s input, expected result, actual result and Pass/Fail status after running the application. Include screenshots of login, student request creation, request timeline, technician updates, admin dashboard, overdue list, category/department management and chatbot responses.

CHAPTER 9 – DEPLOYMENT AND EXECUTION

9.1 Database Setup

From the project's database directory, create the schema and then load sample data. Use the MySQL username and password configured for the local environment.

9.2 Backend Setup

Open a terminal in the backend directory. Configure DB_URL, DB_USER, DB_PASS and JWT_SECRET as environment variables where appropriate, then run the Spring Boot application with Maven. The backend should listen on port 8080. Check the terminal for database connection or compilation errors.

9.3 Frontend Setup

Open a second terminal in the frontend directory, install dependencies with npm install, and start the Vite development server with npm run dev. Open the local URL shown by Vite, normally http://localhost:5173.

9.4 Configuration Check

- The application.properties default JDBC URL uses port 3307; update DB_URL if MySQL is listening on 3306.
- Set DB_PASS to the correct local database password.
- Use a strong, unique JWT_SECRET for non-demo use.
- Ensure ports 8080 and 5173 are available.
- Run schema.sql and seed.sql in the documented order.

9.5 Build Commands

```
Backend: mvn spring-boot:run
Frontend: npm install
Frontend: npm run dev
Frontend production build: npm run build
```

9.6 Demo Accounts

The README lists demo accounts for administrator, technician and student roles. Use these only in a local demonstration environment; change the passwords before exposing the application to other users.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

10.1 Challenges Addressed

- Coordinating request ownership and role-specific actions.
- Maintaining a timeline of status updates.
- Routing requests to departments and technicians.
- Providing a useful category suggestion without a trained model.
- Identifying stale requests that may need escalation.
- Protecting endpoints with token-based authentication.

10.2 Limitations

- The category classifier is keyword-based and can misclassify ambiguous descriptions.
- The FAQ chatbot is limited to the FAQs and keywords available in its data.
- The project’s local configuration includes development defaults that must be changed for production.
- The archive’s supplied documentation does not include evidence of automated test execution or a production deployment.
- Email/SMS notifications, campus map integration, service-level analytics and enterprise SSO are not documented as implemented.

10.3 Risk Mitigation

Use environment-based secrets, enforce least-privilege access, validate inputs, keep database backups, review audit history, test role boundaries and monitor overdue requests. Add automated tests and structured logging before production rollout.

CHAPTER 11 – FUTURE ENHANCEMENTS

11.1 Improved Classification

Replace the keyword scorer with a trained text classifier using labelled historical campus requests. Measure precision, recall and category-wise errors before enabling automatic routing.

11.2 Notifications

Add email, SMS or push notifications for assignment, status changes, overdue requests and request closure.

11.3 Analytics

Provide average resolution time, department workload, repeat issue categories, monthly trends and service-level agreement reports.

11.4 Campus Integration

Integrate campus identity/SSO, building directories, room maps, QR codes and asset-management records.

11.5 Reliability and Operations

Add automated unit/integration tests, structured logging, health checks, database backups, migration tooling, rate limiting and secure deployment configuration.

11.6 Mobile Experience

Improve responsive layouts and consider a progressive web app so students and staff can submit and track requests conveniently from mobile devices.

CHAPTER 12 – CONCLUSION

12.1 Summary

CampusCare provides a centralized approach to campus service management by connecting request submission, role-based assignment, progress tracking, administration and feedback. Its React frontend communicates with a Spring Boot REST API, while MySQL stores the core records and request history.

12.2 Achievements

- Defined a structured request lifecycle.
- Implemented role-aware authentication and REST endpoints.
- Separated core data into related database entities.
- Added keyword-based category suggestion and FAQ matching.
- Included administrative statistics and overdue request detection.
- Provided feedback collection and request history.

12.3 Learning Outcomes

The project demonstrates practical use of frontend/backend separation, REST API design, relational database modelling, ORM repositories, JWT authentication, role-based access, CRUD operations and service workflow design. With additional automated testing and production hardening, the platform can be developed further into a reliable campus operations tool.

CHAPTER 13 – REFERENCES

13.1 Project Sources

- CampusCare Smart Campus Service Management System source archive supplied for analysis.
- Project README.md — stack, setup steps, API endpoints, features and project structure.
- database/schema.sql — relational schema and constraints.
- database/seed.sql — sample departments, categories, users and requests.
- backend/pom.xml — backend dependencies and Java/Spring Boot versions.
- backend/src/main/resources/application.properties — runtime and datasource configuration.

13.2 Technology Documentation

- React documentation.
- Vite documentation.
- Spring Boot reference documentation.
- Spring Security reference documentation.
- Spring Data JPA reference documentation.
- MySQL 8.0 reference manual.
- JSON Web Token (JWT) standards and library documentation.

CHAPTER 14 – APPENDICES

Appendix A – Project Structure

```
smart-campus/
■■■ backend/
■   ■■■ pom.xml
■   ■■■ src/main/java/com/campus/smartcampus/
■       ■■■ config/       AppConfig, SecurityConfig
■       ■■■ model/        User, ServiceRequest, Category, Department,
■       ■                 RequestUpdate, Feedback, ChatbotFaq, Enums
■       ■■■ repo/         Spring Data JPA repositories
■       ■■■ security/     JwtUtil, JwtFilter
■       ■■■ service/      RequestService, ClassifierService
■       ■■■ web/          Auth, Request, Admin, User, Chatbot controllers
■■■ frontend/
■   ■■■ package.json
■   ■■■ src/              App.jsx, api.js, main.jsx, styles.css
■■■ database/
■   ■■■ schema.sql
■   ■■■ seed.sql
■   ■■■ useful_queries.sql
■■■ README.md
```

Appendix B – Important Endpoints

- Authentication: /api/auth/register, /api/auth/login, /api/auth/me
- Requests: /api/requests, /api/requests/{id}, /api/requests/{id}/history
- Workflow: /api/requests/{id}/status, /api/requests/{id}/assign-department, /api/requests/{id}/assign-technician
- Feedback: /api/requests/{id}/feedback
- Admin: /api/admin/stats, /api/admin/overdue, /api/admin/categories, /api/admin/departments
- Users: /api/users, /api/users/technicians
- Chatbot: /api/chatbot/ask?q=..., /api/chatbot/faqs

Appendix C – User Guide

- 1. Start MySQL and initialize the database using schema.sql and seed.sql.
- 2. Configure backend database credentials and JWT secret.
- 3. Start the backend with Maven.
- 4. Install frontend dependencies and start Vite.
- 5. Open the frontend in a browser and sign in with a local demo account.
- 6. As a student, create a request and include an accurate building/floor/room.
- 7. Track the request timeline; technicians/admins update status and assignment.
- 8. After resolution, submit feedback and use the chatbot for common questions.

Appendix D – Documentation Note

This report describes functionality found in the supplied source archive and README. The testing section proposes verification steps and does not claim tests passed unless they are run and results are recorded. Add actual application screenshots and measured test outcomes before final academic submission.